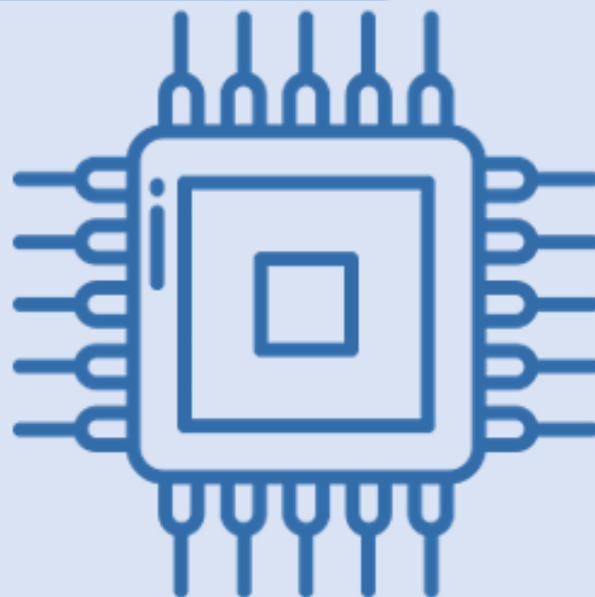


AIN SHAMS UNIVERSITY

Basics of Mechatronics – MCT231



Final Project Team 16



Ziad Atef Mohamed Riad

Kamal Wahdan

Zeyad Amr Nazih

Yahya Ashraf

Abdoulrahman Adel

24P0103

24P0222

24P0372

24P0370

24P0098

Table of Contents	1
1. Introduction	4
1.1 Project Objectives	4
2. Design Thinking Process	5
2.1 User Research	5
2.2 Design Challenge	5
2.3 Ideation & Concept	5
2.4 Iterative Design	5
3. Product Design Specification (PDS)	6
3.1 Performance & Functional Requirements	6
3.2 Electronics & Control System	6
3.3 Physical & Material Specifications	6
3.4 Target Audience	7
4. System Architecture	8
4.1 Block Diagram Overview	8
MCU 1 — Sorting & User Input (ESP32 #1)	8
MCU 2 — Lifting, Shooting & Feedback (ESP32 #2)	8
4.2 System Flowchart	8
5. Mechanical Design	10
5.1 Sorting Mechanism	10
5.2 Lifting Mechanism — Archimedes Screw	10
5.3 Shooting Mechanism	10
5.4 Fully Integrated CAD Assembly	11
6. Electrical Design	12
6.1 Component Summary	12
6.2 Wiring Diagram Overview	12
6.2.1 Sorting Module (MCU1)	12
6.2.2 Lifting Module (MCU1 → A4988 → NEMA 17)	12
6.2.3 Shooting Module (MCU2)	13
6.3 Custom PCB	13
7. Firmware & Control	15
7.1 MCU1 — Sorting & Lifting (ESP32 #1)	15
7.2 MCU2 — Shooting & Feedback (ESP32 #2)	20
7.3 ESP-NOW Communication	28
7.4 User Interfaces	28
Interface 1 — LCD Displays	28
Interface 2 — Processing IO Bluetooth App	28
8. Bill of Materials	29
9. Engineering Calculations	30

9.1 Actuator Sizing — Sorting Mechanism	30
9.2 Actuator Sizing — Lifting Mechanism	30
9.3 Actuator Sizing — Shooting Mechanism.....	30
9.4 Projectile Calculations	31
9.5 Subsystem Current Consumption	31
9.6 Power Source Sizing	32
10. Safety Considerations	33
10.1 Mechanical Safety	33
10.2 Electrical Safety.....	33
10.3 Thermal Safety	33
10.4 Software Safety	33
11. Gantt Chart	34
12. Work Breakdown Structure (WBS).....	35
Phase 1 — Project Management & Preliminary Design	35
Phase 2 — Design Thinking & Ideation	35
Phase 3 — Mechanical Design & Analysis	35
Phase 4 — Mechatronics & Control Systems	35
Phase 5 — Fabrication & Integration.....	35
Phase 6 — Deliverables & Documentation	35
13. Git Repository	36
15. Conclusion	39

1. Introduction

Planet Sort (marketed to children as Orbit Quest) is an interactive, space-themed mechatronic toy designed for children aged 2–10. The toy presents a fully automated ball-sorting and launching system, where coloured planetary marbles are loaded, sorted by colour, lifted vertically via an Archimedes screw, and launched toward a target through a high-velocity shooting mechanism.

The project was developed as part of the MCT231 Basics of Mechatronics final project at Ain Shams University in collaboration with the University of East London. It integrates mechanical design, electronics, embedded firmware, and product design into a cohesive educational toy that fosters engagement, colour recognition, and curiosity in STEM topics.

1.1 Project Objectives

- Design and fabricate a fully functional electromechanical toy that integrates three distinct mechanisms: colour sorting, vertical lifting, and projectile shooting.
- Implement a dual-microcontroller architecture using two ESP32 boards communicating via the ESP-NOW wireless protocol.
- Provide two user interfaces: an onboard LCD display and a Bluetooth-connected Processing IO application.
- Ensure all mechanical, electrical, and firmware subsystems work as a closed-loop marble pathway.
- Document the complete design process from user research to final prototype.

2. Design Thinking Process

2.1 User Research

The target audience was defined as children aged 2–10, with a primary engagement focus on children around 8 years old. User research revealed several key insights:

- Children in this age group are highly motivated by fast-moving, visually stimulating toys.
- Space and planetary themes score very high in engagement for both boys and girls.
- Simple, colour-based interactions are accessible even to younger children, while the automated mechanics provide complexity for older users.
- Safety is paramount: no sharp edges, no exposed wiring, and no small detachable parts that can be swallowed by younger children.

2.2 Design Challenge

The core design challenge was: How can we create a toy that is simultaneously engaging for a broad age range, educational in its interactions, safe by construction, and technically complex enough to demonstrate a complete mechatronic system?

The answer was to build a closed-loop automated machine themed around space and planet sorting, where the child's role is limited to choosing which colour marble to sort and watching the machine respond. This lowers the barrier of entry for younger children while still delighting older ones with the engineering of the system.

2.3 Ideation & Concept

Several concepts were explored via mind maps and mood boards before settling on the Planet Sort concept:

- Option A: Basketball-themed shooter with coloured balls. Relatable but not highly original.
- Option B: Recycling plant (similar to course example). Educational but less engaging visually.
- Option C: Planet Sort — space-themed sorting and launching of planetary marbles (Mars = red, Earth = green). Selected for its strong visual theme, broad age appeal, and alignment with all three required mechanisms.

2.4 Iterative Design

The design went through multiple iterations:

1. Initial sketches established the layout: screw lifter on the left, sorting gate in the middle channel, shooting mechanism at the top right.
2. CAD modelling in Autodesk Inventor refined dimensions and verified fit within the 100 cm × 100 cm constraint.
3. Wiring diagrams were drafted in Cirkuit Designer, separating the three subsystems for modularity.
4. Physical prototyping with 3D-printed parts led to revisions in the screw pitch and the servo gate angle.

3. Product Design Specification (PDS)

3.1 Performance & Functional Requirements

Requirement	Specification
Primary Function	Automated colour-based sorting, vertical lifting and projectile launching of planetary marbles.
Sorting Mechanism	TCS3200 colour sensor detects Red or Green marble. Two SG90 servo-driven gates direct the marble to the correct channel.
Lifting Mechanism	NEMA 17 stepper-driven Archimedes screw lifts marbles from the lower channel to the shooting platform.
Shooting Mechanism	Dual FA-130RA-18100 DC motors spinning rubber-rimmed wheels accelerate the marble to approx. 4.15 m/s at 30° launch angle over a 0.8 m range.
Closed-Loop Path	Single marble entry point; marbles remain within the internal track throughout operation.
User Interaction	Start/Stop button, Red/Green colour selection buttons, and onboard LCD display feedback.

3.2 Electronics & Control System

Parameter	Specification
Microcontrollers	2× ESP32 (DevKitC). MCU1 handles sorting and lifting. MCU2 handles shooting and display.
Communication	ESP-NOW wireless peer-to-peer protocol (low-latency, no router required).
User Interface 1	2× 16×4 I2C LCD displays showing system state, selected colour, and mode (manual / Bluetooth).
User Interface 2	Processing IO Bluetooth application on Android/PC — Start, Stop, colour selection.
Custom PCB	Power distribution board with 12 V input, PTC fuse, reverse-polarity diode, 12 V→5 V buck converter, and modular connectors for all subsystems.
Power Source	Single 12 V DC supply (adapter or 12 V LiPo battery). Four buck converters regulate to 5 V for logic and servos.

3.3 Physical & Material Specifications

Parameter	Specification
Dimensions	Maximum 100 cm × 100 cm footprint. Nominal design: ~70 cm × 60 cm.
Materials	PLA 3D-printed structural and mechanical parts. MDF base board. Smooth surfaces with no sharp edges.
Modularity	Three independent wiring modules (Sorting, Lifting, Shooting) with dedicated JST plug connectors to a central PCB.

Safety	All wiring concealed inside channels. No exposed moving pinch points. Marble diameter >35 mm (not a choking hazard for ages 2+).
---------------	---

3.4 Target Audience

- Primary: Children aged 6–10 years — full autonomous use.
- Secondary: Children aged 2–5 years — supervised play; adult loads marbles and selects colour.
- Tertiary: Parents/educators — observe educational value in STEM, colour recognition, and cause-and-effect reasoning.

4. System Architecture

4.1 Block Diagram Overview

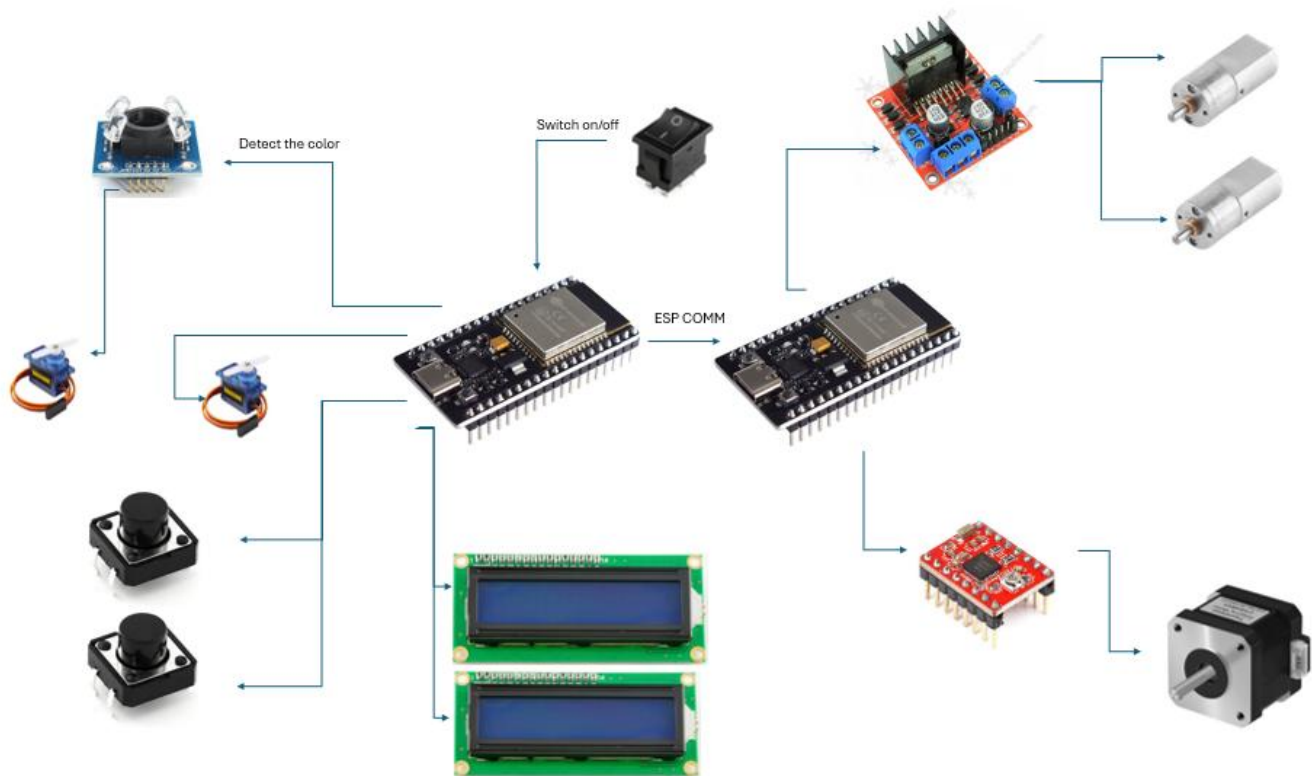
The system is divided into two microcontroller domains connected by ESP-NOW wireless communication:

MCU 1 — Sorting & User Input (ESP32 #1)

- Inputs: TCS3200 colour sensor, Colour 1 button, Colour 2 button, On/Off switch.
- Outputs: Selecting servo (path gate), Sorting servo (end gate), 16×4 I2C LCD display #1.
- Transmits: Colour decision and start/stop command to MCU2 via ESP-NOW.

MCU 2 — Lifting, Shooting & Feedback (ESP32 #2)

- Inputs: ESP-NOW message from MCU1.
- Outputs: A4988 stepper driver → NEMA 17 (Archimedes screw), L298N H-bridge → 2× DC motors (shooting wheels), 16×4 I2C LCD display #2.

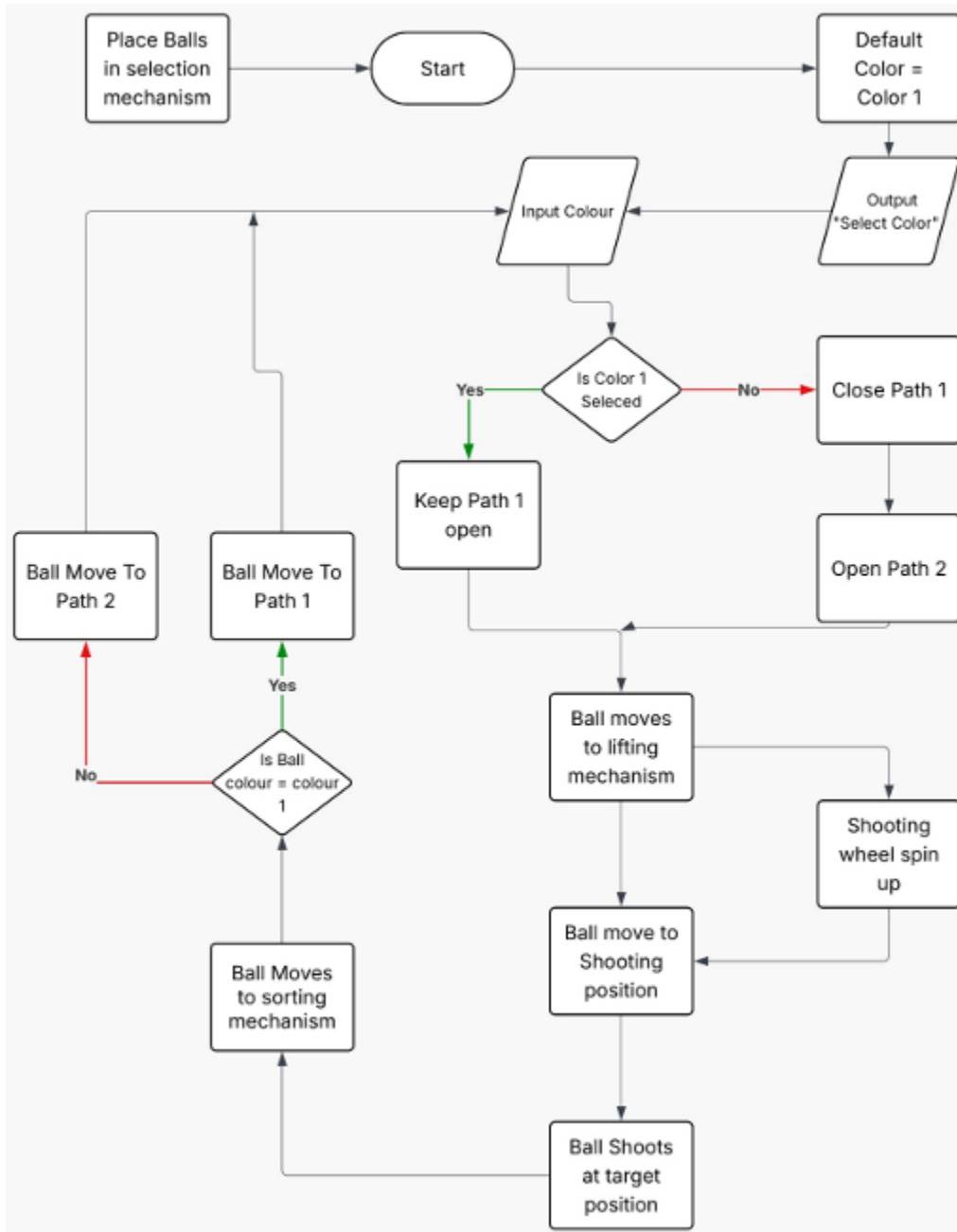


4.2 System Flowchart

The following describes the operational logic of the complete system:

5. Balls are placed into the selection mechanism (entry point). System starts on button press.
6. Default colour is set to Colour 1 (Green). LCD outputs 'Select Color'.
7. User presses Green or Red button (or selects via Bluetooth app).
8. Is Color 1 (Green) selected? If YES: Path 1 stays open. If NO: Path 1 closes, Path 2 opens.
9. Ball moves through the selected path to the sorting mechanism.

10. Colour sensor reads the ball. Is ball colour = selected colour? If YES: Ball moves to lifting mechanism. If NO: Ball returns to sorting mechanism.
11. Ball is lifted by Archimedes screw to shooting platform.
12. Shooting wheels spin up to operating speed (~1,743 rpm).
13. Ball moves to shooting position and is launched toward target.
14. Cycle completes; system awaits next ball or Stop command.

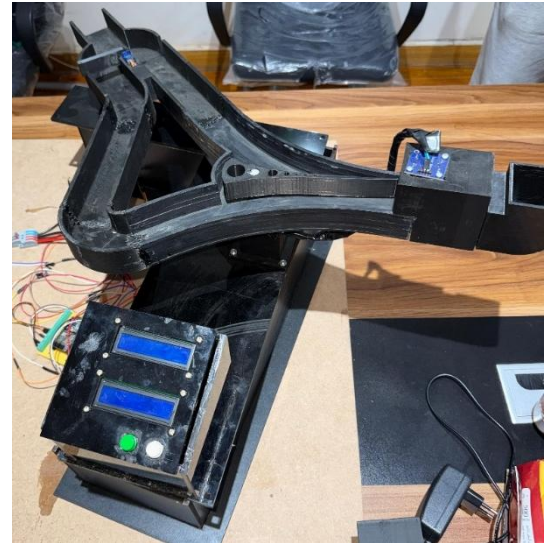


5. Mechanical Design

5.1 Sorting Mechanism

The sorting mechanism uses two SG90 micro servo motors to actuate gate flaps within the marble channel:

- **Selecting Servo:** Acts as the initial path router. When Color 1 is selected, the flap remains in position to keep Path 1 open. When Color 2 is selected, the flap rotates to close Path 1 and open Path 2.
- **Sorting Servo (End Gate):** Positioned at the output of the colour-sensing zone. After the TCS3200 confirms the correct colour, this servo opens to release the marble to the lifting mechanism. If the colour does not match, it redirects the marble back.
- **CAD:** Both gate mechanisms are 3D-printed in PLA. Servo horn connectors are press-fit with M2 screws for easy replacement.



5.2 Lifting Mechanism — Archimedes Screw

The vertical lifting mechanism is based on an Archimedes screw — a helical blade wound around a central shaft enclosed in a smooth cylindrical tube:

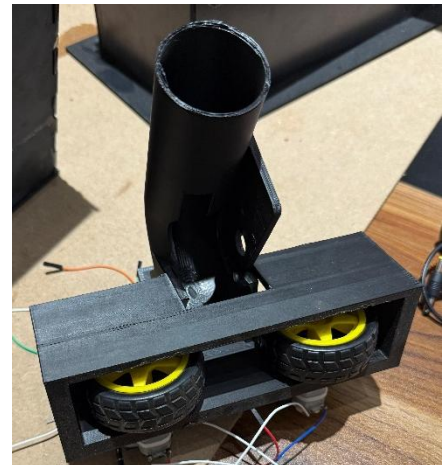
- **Motor:** NEMA 17 stepper motor (operating at ~500 rpm) drives the screw shaft via a direct-drive coupler.
- **Screw dimensions:** Pitch and diameter sized to accept marbles of 35–40 mm diameter and transport one marble per revolution reliably.
- **The enclosed tube** prevents marbles from falling sideways and contributes to the closed-loop pathway requirement.
- **All parts 3D-printed in PLA.** The screw was printed in sections and assembled on the shaft.



5.3 Shooting Mechanism

The shooting mechanism uses a dual-wheel friction launcher, widely used in table tennis ball machines and youth sports equipment:

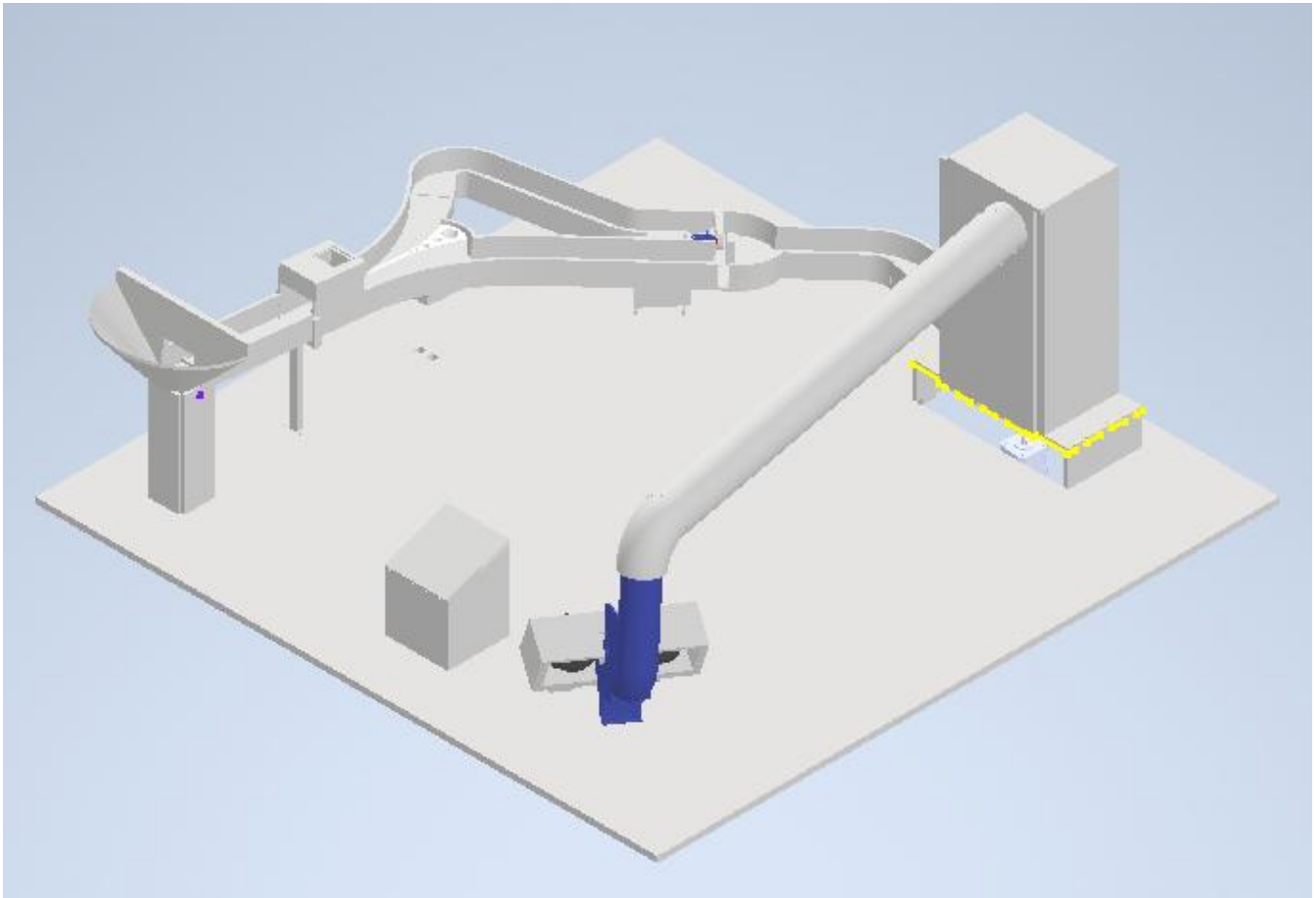
- **Two FA-130RA-18100 DC motors** each drive a rubber-rimmed PLA wheel (~65 mm diameter, 35 g).
- The wheels are arranged facing each other with a gap slightly smaller than the marble diameter, creating a compression grip.
- When the marble is fed between the spinning wheels, friction accelerates it to the calculated exit velocity (~4.15 m/s).
- **Motor driver:** L298N H-bridge controlled via PWM from MCU2, allowing speed adjustment.
- **Launch angle:** 30° above horizontal, achieving a 0.8 m horizontal range at a 0.2 m height difference to the target hoop.



5.4 Fully Integrated CAD Assembly

The complete assembly was modelled in Autodesk Inventor. Key design decisions:

- Base plate: 700 mm × 600 mm MDF, within the 100 cm × 100 cm constraint.
- The Archimedes screw tower is positioned at the rear-left corner with height ~350 mm.
- The ball channel runs along the base from the entry point (front-right) through the sorting zone to the screw base (rear-left) and back to the shooter (top of screw, directed right).
- The target hoop is mounted at the rear-right, aligned with the 30° shooting trajectory.
- Wire routing channels are integrated into all structural parts to keep wiring completely hidden.



6. Electrical Design

6.1 Component Summary

#	Component	Role	Interface	Qty
1	ESP32 DevKitC	Main microcontrollers	GPIO, I2C, PWM	2
2	TCS3200 Colour Sensor	Ball colour detection	Digital GPIO	1
3	SG90 Micro Servo	Sorting & selecting gates	PWM	2
4	NEMA 17 Stepper Motor	Archimedes screw drive	A4988 driver	1
5	A4988 Stepper Driver	NEMA 17 control	STEP/DIR PWM	1
6	FA-130RA-18100 DC Motor	Shooting wheels	L298N H-Bridge	2
7	L298N H-Bridge Module	DC motor drive	PWM GPIO	1
8	16×4 I2C LCD Display	User feedback	I2C (SDA/SCL)	2
9	12 V→5 V Buck Converter	Logic/servo power	Power rail	4
10	Custom PCB (Power Dist.)	Power routing, protection	Terminal blocks	1
11	Push Buttons (12 mm)	Colour select, Start/Stop	Digital GPIO	4
12	Rocker Switch	Main power On/Off	Power rail	1
13	12 V DC Power Supply / LiPo	Single power source	—	1

6.2 Wiring Diagram Overview

The electrical system is divided into three independent wiring modules. Each module has its own cable harness terminated with JST connectors, which plug into the central custom PCB for easy assembly and maintenance.

6.2.1 Sorting Module (MCU1)

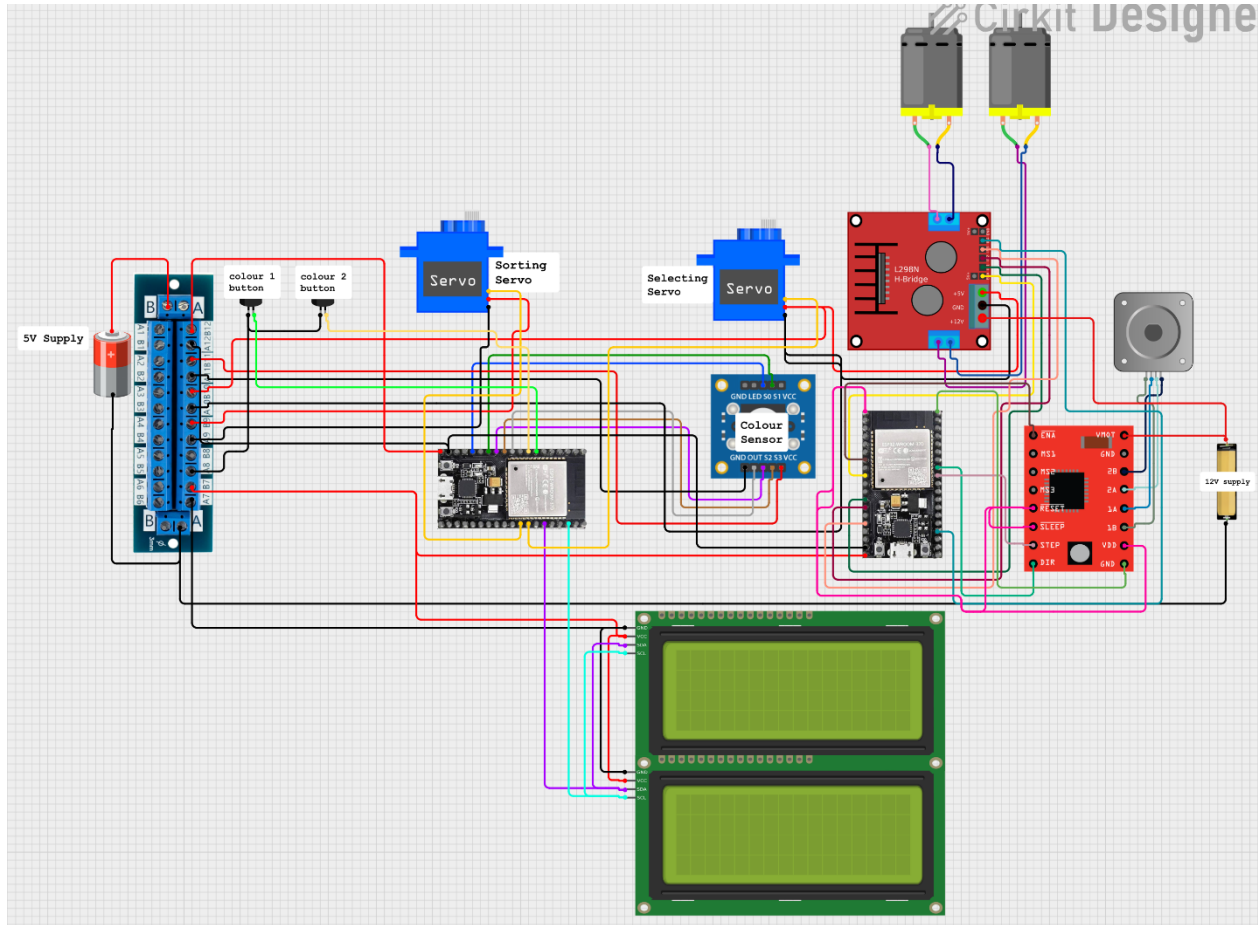
- Arduino 1 (ESP32) ↔ TCS3200: S0, S1, S2, S3 (GPIO outputs), OUT (GPIO input), LED enable, VCC/GND.
- Arduino 1 ↔ Selecting Servo: PWM signal + 5 V / GND.
- Arduino 1 ↔ Sorting Servo: PWM signal + 5 V / GND.
- Arduino 1 ↔ LCD Display #1: SDA/SCL (I2C), 5 V / GND.
- Colour 1 & Colour 2 push-buttons: pulled up internally, connected to GPIO with GND.
- 5 V supply via terminal block from PCB buck converter.

6.2.2 Lifting Module (MCU1 → A4988 → NEMA 17)

- Arduino 1 ↔ A4988: STEP (PWM), DIR (GPIO), ENA (GPIO), MS1/MS2/MS3 (microstepping select), SLEEP/RESET.
- A4988 VMOT: 12 V from PCB. VDD: 5 V from buck. B1/B2/A1/A2: NEMA 17 coil pairs.
- 100 μ F decoupling capacitor across VMOT/GND of A4988.

6.2.3 Shooting Module (MCU2)

- Arduino 2 (ESP32) ↔ L298N: ENA (PWM), IN1/IN2 (direction), ENB (PWM), IN3/IN4 (direction).
- L298N OUT1/OUT2 → DC Motor Left (shooting wheel). OUT3/OUT4 → DC Motor Right.
- L298N +12 V from PCB, +5 V from on-board regulator. GND common.
- Arduino 2 ↔ LCD Display #2: SDA/SCL (I2C), 5 V / GND.

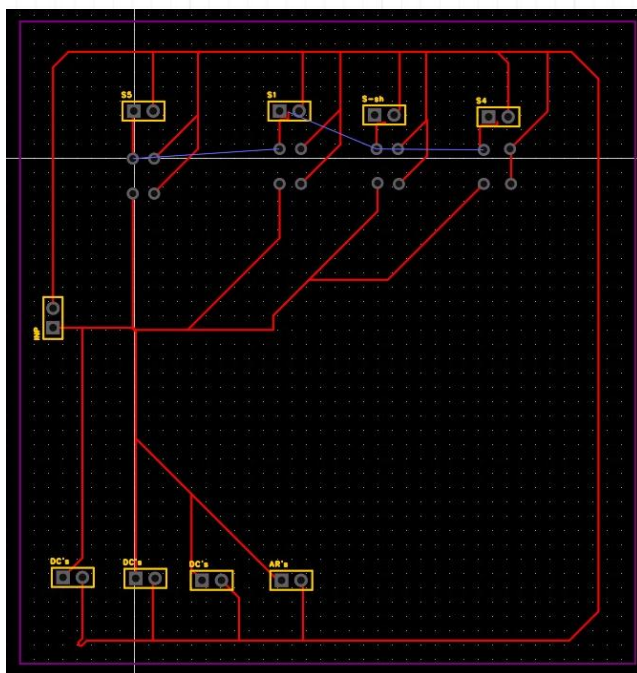
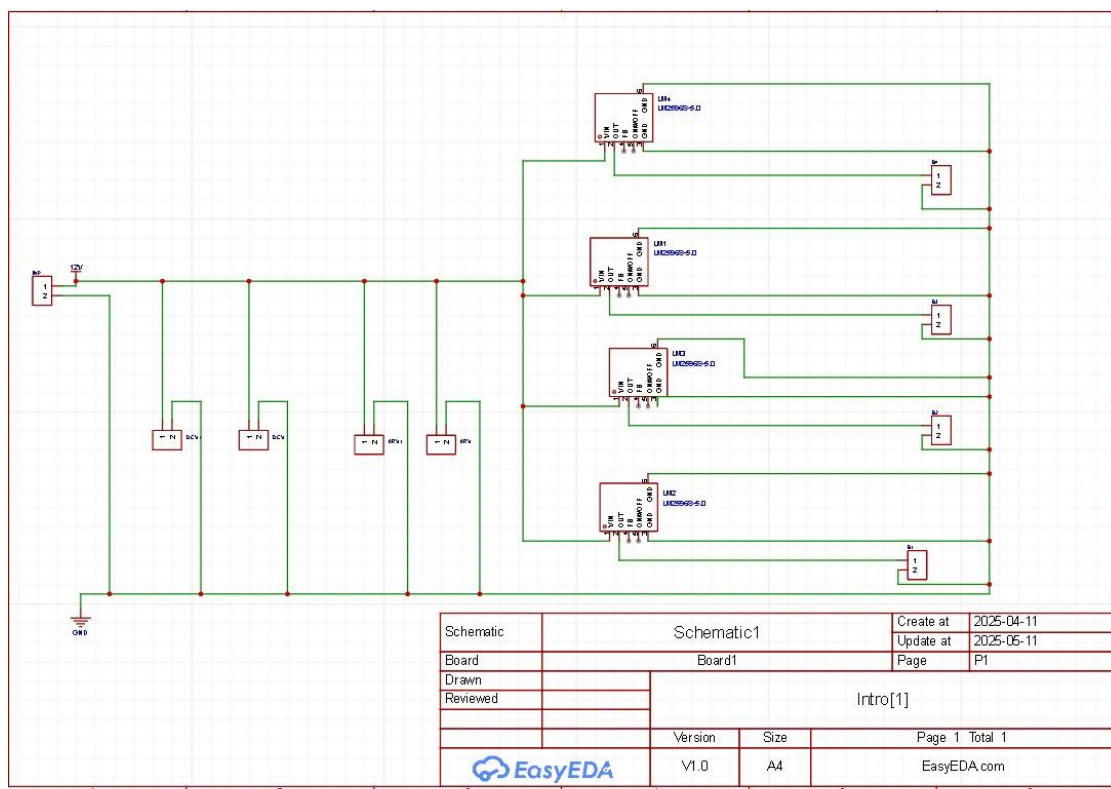


<https://app.cirkitdesigner.com/project/e711d32f-553b-48ab-a936-7aebbb711319>

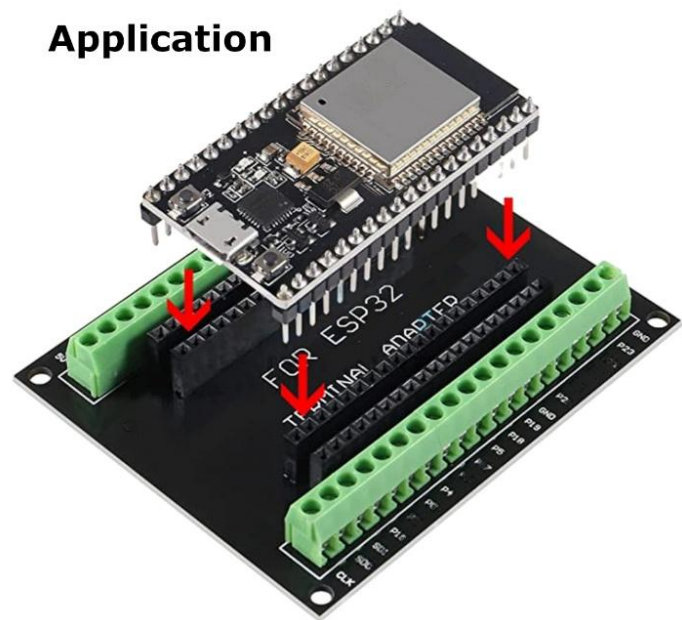
6.3 Custom PCB

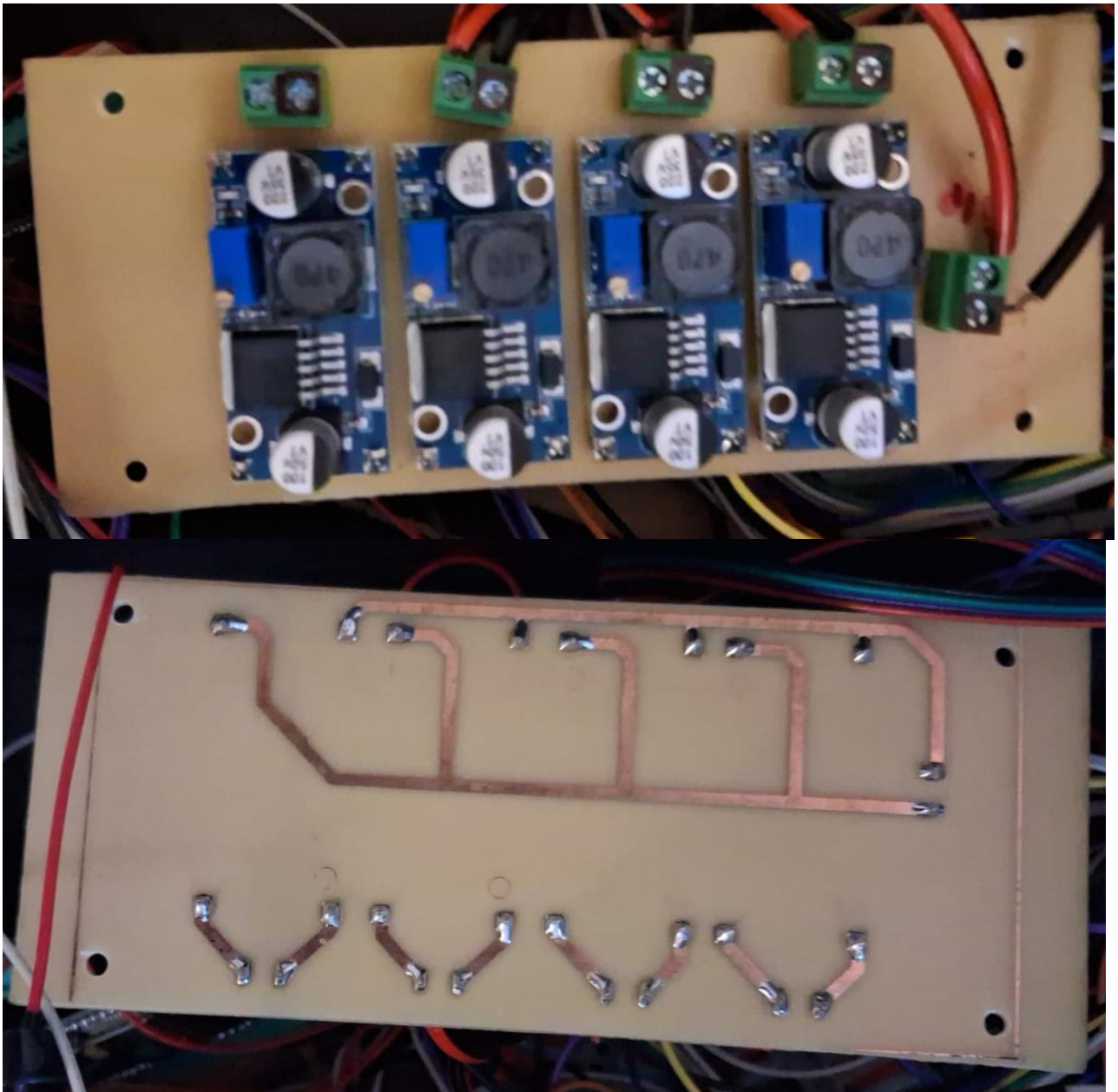
The custom PCB serves as the power distribution and protection hub for the entire toy. Key features:

- Input: 2-pin JST connector (J2) accepting 12 V DC.
- Reverse-polarity protection: 1N5822 Schottky diode (D1).
- Over-current protection: PTC resettable fuse (F1) rated at 5 A.
- 12 V → 5 V Buck converter (LM2596-based) with 100 μ F input and 1000 μ F output capacitors, 10 μ H inductor.
- Output connectors: Servo mover 1, Servo end gate, Right motor, Left motor, Stepper 1, Arduino R4 (5 V regulated).
- Ground plane poured across the full board for noise immunity.



Application





7. Firmware & Control

7.1 MCU1 — Sorting & Lifting (ESP32 #1)

MCU1 is responsible for all input handling and the sorting and lifting subsystems. The firmware structure is:

- Initialisation: Set up I2C LCD, configure TCS3200 frequency pins, attach servo objects, configure button GPIO with internal pull-ups.
- Main Loop: Poll colour selection buttons. If pressed, update selectedColour variable and display on LCD.
- On Start command: Actuate Selecting Servo to open correct path. Read TCS3200 output. If colour matches selectedColour, release Sorting Servo gate, then send ESP-NOW trigger message to MCU2 to run the lifting sequence.

- ESP-NOW Transmit: MCU1 broadcasts a struct containing {command: LIFT_AND_SHOOT, colour: selected} to MCU2 MAC address.
- LCD Display: Shows 'SYSTEM: READY', 'COLOR: GREEN', 'MODE: MANUAL' etc. based on state machine.

Here is the code:

```
/*
 * =====
 * ESP1 — SLAVE SUBSYSTEM (Lifter & Shooter)
 * MAC: 5C:01:3B:34:3A:64
 *
 * Responsibilities:
 * - Wait for IR_BOTTOM → lift ball with stepper motor
 * - Wait for IR_TOP → fire shooting motors
 * - Receive selectedColour from ESP2 via ESP-NOW
 * - Broadcast real-time status strings back to ESP2
 *   so ESP2 can display them on LCD2 (0x27)
 *
 * ESP-NOW Peer: ESP2 → 3C:8A:1F:AF:CF:98
 * =====
 */

#include <esp_now.h>
#include <WiFi.h>

// — STEPPER (A4988 / DRV8825) —————
#define STEP_PIN 18
#define DIR_PIN 19
#define ENABLE_PIN 32 // LOW = enabled, HIGH = disabled

// — IR SENSORS (LOW = ball detected) —————
#define IR_BOTTOM 27
#define IR_TOP 33

// — SHOOTING MOTORS (L298N) —————
#define IN1 14
#define IN2 12
#define IN3 13
#define IN4 15
#define ENA 25
#define ENB 26

// — SHOOTING SPEED (0–255) —————
int shootSpeed = 200;

// — STATE —————
bool lifting = false;
bool shooting = false;
```

```
unsigned long shootStartTime = 0;
const unsigned long SHOOT_DURATION_MS = 1000; // motors run 1 s
```

```
// — ESP-NOW —————
uint8_t esp2MAC[] = {0x3C, 0x8A, 0x1F, 0xAF, 0xCF, 0x98};
```

```
// Incoming: colour chosen on ESP2
typedef struct IncomingColour {
    uint8_t selectedColour; // 1=RED 2=GREEN 0=none
} IncomingColour;
```

```
// Outgoing: status lines for ESP2's LCD2
typedef struct StatusMsg {
    char line1[17];
    char line2[17];
} StatusMsg;
```

```
IncomingColour receivedData;
volatile bool newColourReceived = false;
uint8_t currentColour = 0;
```

```
// — HELPERS —————
void sendStatus(const char *l1, const char *l2) {
    StatusMsg msg;
    strncpy(msg.line1, l1, 16); msg.line1[16] = '\0';
    strncpy(msg.line2, l2, 16); msg.line2[16] = '\0';
    esp_now_send(esp2MAC, (uint8_t *)&msg, sizeof(msg));
    Serial.printf("[TX Status] '%s' | '%s'\n", msg.line1, msg.line2);
}
```

```
void stopShooter() {
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}
```

```
void startShooter() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    analogWrite(ENA, shootSpeed);
    analogWrite(ENB, shootSpeed);
}
```

```
void enableStepper() { digitalWrite(ENABLE_PIN, LOW); }
```

```
void disableStepper() { digitalWrite(ENABLE_PIN, HIGH); }
```

```
void stepOnce() {  
    digitalWrite(STEP_PIN, HIGH); delayMicroseconds(1500);  
    digitalWrite(STEP_PIN, LOW); delayMicroseconds(1500);  
}
```

```
// — ESP-NOW CALLBACKS —————
```

```
void onDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {  
    // silent — status TX confirmations not needed  
}
```

```
void onDataReceived(const esp_now_recv_info_t *info, const uint8_t *data, int len) {  
    if (len == sizeof(IncomingColour)) {  
        memcpy(&receivedData, data, sizeof(IncomingColour));  
        newColourReceived = true;  
    }  
}
```

```
// =====
```

```
void setup() {  
    Serial.begin(115200);  
    delay(500);
```

```
// — STEPPER —————
```

```
pinMode(STEP_PIN, OUTPUT);  
pinMode(DIR_PIN, OUTPUT);  
pinMode(ENABLE_PIN, OUTPUT);  
digitalWrite(DIR_PIN, LOW);  
disableStepper(); // off at boot
```

```
// — IR SENSORS —————
```

```
pinMode(IR_BOTTOM, INPUT);  
pinMode(IR_TOP, INPUT);
```

```
// — SHOOTER MOTORS —————
```

```
pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);  
pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);  
pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);  
stopShooter();
```

```
// — ESP-NOW —————
```

```
WiFi.mode(WIFI_STA);  
Serial.print("ESP1 MAC: "); Serial.println(WiFi.macAddress());
```

```
if (esp_now_init() != ESP_OK) {
```

```
    Serial.println("[ERROR] ESP-NOW init failed");
    return;
}
esp_now_register_send_cb(onDataSent);
esp_now_register_rcv_cb(onDataReceived);

esp_now_peer_info_t peer = {};
memcpy(peer.peer_addr, esp2MAC, 6);
peer.channel = 0;
peer.encrypt = false;
if (esp_now_add_peer(&peer) != ESP_OK) {
    Serial.println("[ERROR] Failed to add ESP2 peer");
}

setStatus("ESP1 ONLINE   ", "Awaiting ball...");
Serial.println("--- ESP1 Slave Ready ---");
}

// =====
void loop() {
    unsigned long now = millis();

    // — HANDLE NEW COLOUR FROM ESP2 —————
    if (newColourReceived) {
        newColourReceived = false;
        currentColour = receivedData.selectedColour;
        const char *colName = (currentColour == 1) ? "RED" : (currentColour == 2) ? "GREEN" : "NONE";
        Serial.printf("[RX] Colour received: %s\n", colName);

        char l2[17];
        snprintf(l2, sizeof(l2), "Target: %-6s ", colName);
        setStatus("SYSTEM STATUS: ", l2);
    }

    // =====
    // LIFTING — triggered by IR_BOTTOM
    // =====
    int bottomSensor = digitalRead(IR_BOTTOM);
    int topSensor    = digitalRead(IR_TOP);

    if (bottomSensor == LOW && !lifting && !shooting) {
        lifting = true;
        enableStepper();
        Serial.println("[IR_BOTTOM] Ball detected → lifting");
        setStatus("SYSTEM STATUS: ", "Lifting ball... ");
    }
}
```

```
if (lifting) {
    stepOnce(); // non-blocking single step each loop pass
}

// =====
// TOP REACHED — stop lift, fire shooter
// =====
if (topSensor == LOW && lifting) {
    lifting = false;
    disableStepper();
    Serial.println("[IR_TOP] Ball at top → shooting");

    shooting = true;
    shootStartTime = now;
    startShooter();

    // Report which colour is being shot
    const char *colName = (currentColour == 1) ? "RED" : (currentColour == 2) ? "GREEN" : "BALL";
    char l2[17];
    snprintf(l2, sizeof(l2), "Shooting %-6s ", colName);
    sendStatus("SYSTEM STATUS: ", l2);
}

// =====
// SHOOT TIMEOUT — stop motors after SHOOT_DURATION_MS
// =====
if (shooting && (now - shootStartTime >= SHOOT_DURATION_MS)) {
    shooting = false;
    stopShooter();
    Serial.println("[SHOOT] Done → idle");
    sendStatus("SYSTEM STATUS: ", "Awaiting ball...");
}
}
```

7.2 MCU2 — Shooting & Feedback (ESP32 #2)

MCU2 runs the stepper for lifting and the DC motors for shooting, coordinated by incoming ESP-NOW messages.

- On receive ESP-NOW LIFT_AND_SHOOT: Step NEMA 17 at 500 rpm for calculated number of steps to raise marble to shooting platform.
- Spin-up: Both DC motors ramped to full PWM (~1,743 rpm wheel speed) over 3 seconds via PWM increase.
- Feed: After spin-up, a brief servo or stepper pulse feeds the marble between the wheels.
- Shoot: Marble exits at calculated velocity; motors ramp down.
- LCD Display #2: Reflects received colour and mode in sync with MCU1.

Here is the code:

```
/*  
* =====  
* ESP2 — MASTER CONTROLLER (Color Sorter & Brain)  
* MAC: 3C:8A:1F:AF:CF:98  
*  
* Responsibilities:  
* - Read color sensor → sort balls via servo  
* - Read RED/GREEN buttons → manage 20s lock timer  
* - LCD1 (0x23) = User Interface (colour choice, countdown)  
* - LCD2 (0x27) = System Status (ESP1 operational feedback)  
* - Send selectedColour to ESP1 via ESP-NOW  
* - Receive status string from ESP1 and display on LCD2  
*  
* ESP-NOW Peer: ESP1 → 5C:01:3B:34:3A:64  
* =====  
*/
```

```
#include <Wire.h>  
#include <LiquidCrystal_I2C.h>  
#include <ESP32Servo.h>  
#include <esp_now.h>  
#include <WiFi.h>
```

```
// — I2C —————  
#define SDA_PIN 21  
#define SCL_PIN 22
```

```
// — COLOR SENSOR —————  
#define S0 13  
#define S1 12  
#define S2 14  
#define S3 27  
#define sensorOut 26
```

```
// — SERVOS —————  
#define SERVO_COLOR_PIN 19 // sorts balls  
#define SERVO_BTN_PIN 18 // mirrors button selection
```

```
// — BUTTONS (pressed = LOW via INPUT_PULLUP) —————  
#define BTN_RED 32  
#define BTN_GREEN 33
```

```
// — LCDs —————  
LiquidCrystal_I2C lcdUI (0x23, 16, 2); // LCD1 — user interface  
LiquidCrystal_I2C lcdStatus(0x27, 16, 2); // LCD2 — ESP1 status
```

```
// — SERVOS —————  
Servo myServo;    // color sorting (pin 19)  
Servo selectServo; // button mirror (pin 18)  
  
// — COLOR SENSOR READINGS —————  
int freqR = 0, freqG = 0, freqB = 0;  
  
// — ESP-NOW —————  
uint8_t esp1MAC[] = {0x5C, 0x01, 0x3B, 0x34, 0x3A, 0x64};  
  
// Outgoing: tells ESP1 which colour was selected  
typedef struct OutgoingMsg {  
    uint8_t selectedColour; // 0=none 1=RED 2=GREEN  
} OutgoingMsg;  
  
// Incoming: operational status from ESP1  
typedef struct IncomingStatus {  
    char line1[17]; // up to 16 chars + null  
    char line2[17];  
} IncomingStatus;  
  
OutgoingMsg toSend;  
IncomingStatus esp1Status;  
volatile bool newStatusFromESP1 = false;  
  
// — STATE MACHINE —————  
enum SystemState { AWAITING_SELECTION, LOCKED, SELECTING, SORTING };  
SystemState currentState = AWAITING_SELECTION;  
  
int selectedColour = 0;  
  
// — TIMERS —————  
const unsigned long LOCK_DURATION_MS  = 20000;  
const unsigned long COLOR_CHECK_MS    = 1000;  
const unsigned long UI_REFRESH_MS     = 500;  
  
unsigned long lockStartTime    = 0;  
unsigned long lastColorCheck   = 0;  
unsigned long lastUIRefresh    = 0;  
unsigned long sortingDisplayUntil = 0; // hold sorting msg on LCD2 for 3s  
int          lastCountdownShown = -1;  
  
// — ESP-NOW CALLBACKS —————  
void onDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {  
    Serial.print("[ESP-NOW TX] ");  
    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "OK" : "FAIL");
```

```
}

void onDataReceived(const esp_now_rcv_info_t *info, const uint8_t *data, int len) {
    if (len == sizeof(IncomingStatus)) {
        memcpy(&esp1Status, data, sizeof(IncomingStatus));
        newStatusFromESP1 = true;
    }
}
```

```
// — SEND COLOUR TO ESP1 —————
```

```
void sendColourToESP1() {
    toSend.selectedColour = (uint8_t)selectedColour;
    esp_now_send(esp1MAC, (uint8_t *)&toSend, sizeof(toSend));
    Serial.printf("[TX] Sent colour %d to ESP1\n", selectedColour);
}
```

```
// =====
```

```
void setup() {
    Serial.begin(115200);
    delay(500);
```

```
// — I2C + LCDs —————
```

```
Wire.begin(SDA_PIN, SCL_PIN, 100000);
```

```
lcdUI.init(); lcdUI.backlight(); lcdUI.clear();
lcdStatus.init(); lcdStatus.backlight(); lcdStatus.clear();
```

```
lcdUI.setCursor(0,0); lcdUI.print("Welcome User! ");
lcdUI.setCursor(0,1); lcdUI.print("Select: R or G ");
```

```
lcdStatus.setCursor(0,0); lcdStatus.print("SYSTEM STATUS: ");
lcdStatus.setCursor(0,1); lcdStatus.print("Waiting ESP1... ");
```

```
// — COLOR SENSOR —————
```

```
pinMode(S0, OUTPUT); pinMode(S1, OUTPUT);
pinMode(S2, OUTPUT); pinMode(S3, OUTPUT);
pinMode(sensorOut, INPUT);
digitalWrite(S0, LOW); digitalWrite(S1, HIGH); // 20% scaling
```

```
// — BUTTONS —————
```

```
pinMode(BTN_RED, INPUT_PULLUP);
pinMode(BTN_GREEN, INPUT_PULLUP);
```

```
// — SERVOS —————
```

```
ESP32PWM::allocateTimer(0);
ESP32PWM::allocateTimer(1);
```

```
myServo.setPeriodHertz(50);
myServo.attach(SERVO_COLOR_PIN, 500, 2400);
myServo.write(78);
```

```
selectServo.setPeriodHertz(50);
selectServo.attach(SERVO_BTN_PIN, 500, 2400);
selectServo.write(90);
```

```
// — ESP-NOW —————
WiFi.mode(WIFI_STA);
Serial.print("ESP2 MAC: "); Serial.println(WiFi.macAddress());
```

```
if (esp_now_init() != ESP_OK) {
    Serial.println("[ERROR] ESP-NOW init failed");
    lcdStatus.setCursor(0,1); lcdStatus.print("ESP-NOW FAILED! ");
    return;
}
esp_now_register_send_cb(onDataSent);
esp_now_register_recv_cb(onDataReceived);
```

```
esp_now_peer_info_t peer = {};
memcpy(peer.peer_addr, esp1MAC, 6);
peer.channel = 0;
peer.encrypt = false;
if (esp_now_add_peer(&peer) != ESP_OK) {
    Serial.println("[ERROR] Failed to add ESP1 peer");
}
```

```
Serial.println("--- ESP2 Master Ready ---");
}
```

```
// =====
void loop() {
    unsigned long now = millis();
```

```
// — UPDATE LCD2 WITH ESP1 STATUS —————
// Do not overwrite LCD2 while ESP2 itself is actively sorting a ball
if (newStatusFromESP1 && currentState != SORTING) {
    newStatusFromESP1 = false;
    lcdStatus.clear();
    lcdStatus.setCursor(0,0); lcdStatus.print(esp1Status.line1);
    lcdStatus.setCursor(0,1); lcdStatus.print(esp1Status.line2);
}
```

```
// =====
// STATE 1: AWAITING FIRST COLOUR SELECTION
```

```
// =====  
if (currentState == AWAITING_SELECTION) {  
    if (digitalRead(BTN_RED) == LOW) {  
        selectedColour = 1;  
        selectServo.write(180);  
        currentState = LOCKED;  
        lockStartTime = now;  
        lastCountdownShown = -1;  
        sendColourToESP1();  
        delay(200);  
    }  
    else if (digitalRead(BTN_GREEN) == LOW) {  
        selectedColour = 2;  
        selectServo.write(0);  
        currentState = LOCKED;  
        lockStartTime = now;  
        lastCountdownShown = -1;  
        sendColourToESP1();  
        delay(200);  
    }  
}  
  
// =====  
// STATE 2: LOCKED — countdown on LCD1, buttons ignored  
// =====  
if (currentState == LOCKED) {  
    unsigned long elapsed = now - lockStartTime;  
  
    if (elapsed >= LOCK_DURATION_MS) {  
        currentState = SELECTING;  
        lastCountdownShown = -1;  
        Serial.println("[LOCK] Expired — buttons unlocked.");  
        lcdUI.clear();  
        lcdUI.setCursor(0,0); lcdUI.print("Change colour? ");  
        lcdUI.setCursor(0,1); lcdUI.print("Press RED or GRN");  
    }  
    else {  
        int secsLeft = (int)((LOCK_DURATION_MS - elapsed + 999) / 1000);  
        if (secsLeft != lastCountdownShown) {  
            lastCountdownShown = secsLeft;  
            lcdUI.clear();  
            lcdUI.setCursor(0,0);  
            lcdUI.print(selectedColour == 1 ? "Colour: RED" : "Colour: GREEN ");  
            lcdUI.setCursor(0,1);  
            char buf[17];  
            snprintf(buf, sizeof(buf), "Locked [%2ds] ", secsLeft);  
        }  
    }  
}
```

```
    lcdUI.print(buf);
  }
}
}

// =====
// STATE 3 / 4: SELECTING / SORTING — post-lock button input
// =====
if (currentState == SELECTING || currentState == SORTING) {
  if (digitalRead(BTN_RED) == LOW) {
    selectedColour = 1;
    selectServo.write(180);
    sendColourToESP1();
    lcdUI.clear();
    lcdUI.setCursor(0,0); lcdUI.print("Colour: RED  ");
    lcdUI.setCursor(0,1); lcdUI.print("          ");
    delay(200);
  }
  else if (digitalRead(BTN_GREEN) == LOW) {
    selectedColour = 2;
    selectServo.write(0);
    sendColourToESP1();
    lcdUI.clear();
    lcdUI.setCursor(0,0); lcdUI.print("Colour: GREEN ");
    lcdUI.setCursor(0,1); lcdUI.print("          ");
    delay(200);
  }
}

// =====
// COLOR SENSOR — non-blocking, every 1 second
// =====
if (now - lastColorCheck >= COLOR_CHECK_MS) {
  lastColorCheck = now;

  digitalWrite(S2, LOW); digitalWrite(S3, LOW);
  freqR = pulseIn(sensorOut, LOW); delay(10);

  digitalWrite(S2, HIGH); digitalWrite(S3, HIGH);
  freqG = pulseIn(sensorOut, LOW); delay(10);

  digitalWrite(S2, LOW); digitalWrite(S3, HIGH);
  freqB = pulseIn(sensorOut, LOW); delay(10);

  Serial.printf("Sensor R=%d G=%d B=%d\n", freqR, freqG, freqB);
```

```
if (freqR < freqG && freqR < 35) {
  myServo.write(68);
  Serial.println("-> RED ball detected");
  currentState = SORTING;
  sortingDisplayUntil = millis() + 3000;
  lcdStatus.clear();
  lcdStatus.setCursor(0,0); lcdStatus.print("SORTING BALL... ");
  lcdStatus.setCursor(0,1); lcdStatus.print("Detected: RED ");
}
else if (freqG < freqR && freqG < 30) {
  myServo.write(88);
  Serial.println("-> GREEN ball detected");
  currentState = SORTING;
  sortingDisplayUntil = millis() + 3000;
  lcdStatus.clear();
  lcdStatus.setCursor(0,0); lcdStatus.print("SORTING BALL... ");
  lcdStatus.setCursor(0,1); lcdStatus.print("Detected: GREEN ");
}
else {
  myServo.write(78);
  if (currentState == SORTING && millis() >= sortingDisplayUntil) {
    currentState = SELECTING;
    lcdStatus.clear();
    lcdStatus.setCursor(0,0); lcdStatus.print("SYSTEM STATUS: ");
    lcdStatus.setCursor(0,1); lcdStatus.print("Selecting... ");
  }
}
}

// =====
// IDLE LCD1 REFRESH (rate-limited, no flicker)
// =====

if (currentState != LOCKED && (now - lastUIRefresh >= UI_REFRESH_MS)) {
  lastUIRefresh = now;
  if (currentState == AWAITING_SELECTION) {
    lcdUI.setCursor(0,0); lcdUI.print("Welcome User! ");
    lcdUI.setCursor(0,1); lcdUI.print("Select: R or G ");
  }
  else if (currentState == SELECTING) {
    lcdUI.setCursor(0,0);
    if (selectedColour == 1) lcdUI.print("Colour: RED ");
    else if (selectedColour == 2) lcdUI.print("Colour: GREEN ");
    else
      lcdUI.print("Change colour? ");
    lcdUI.setCursor(0,1); lcdUI.print("Press RED or GRN");
  }
}
```

```
}
```

7.3 ESP-NOW Communication

ESP-NOW is an Espressif peer-to-peer wireless protocol that operates without a Wi-Fi router. Advantages for this application:

- Latency: Sub-millisecond typical message delivery — essential for real-time marble coordination.
- No network dependency: The toy works anywhere without internet or a router.
- Simple API: `espnow_send()` / `espnow_rcv_cb()` used directly in Arduino framework.
- Packet structure: 8-byte struct `{uint8_t cmd; uint8_t colour; uint8_t reserved[6]}`.

7.4 User Interfaces

Interface 1 — LCD Displays

Two 16×4 character LCD displays with I2C backpacks (PCF8574 expander) are connected to MCU1 and MCU2 respectively. Display states:

- SYSTEM: READY — after power on, before start.
- COLOR: GREEN / RED — shows selected colour after button press.
- MODE: MANUAL / BT — indicates whether input came from physical button or Bluetooth.
- SORTING... / LIFTING... / SHOOTING... — real-time status during operation.

Interface 2 — Processing IO Bluetooth App

A Processing IO application running on a smartphone or PC communicates with MCU1 via Bluetooth (HC-05 module or ESP32 built-in Bluetooth Serial). The app provides:

- CONNECT button: Establishes Bluetooth serial link.
- START / STOP buttons: Send 'S' and 'X' characters to MCU1.
- GREEN / RED planet buttons: Send 'G' and 'R' characters to set colour selection remotely.
- Status panel: Receives and displays MCU1 serial responses ('READY', 'SORTING', 'DONE', etc.).

8. Bill of Materials

#	Component	Specification / Model	Qty	Unit (EGP)	Total (EGP)	Subsystem
1	ESP32 DevKitC	Espressif ESP32-WROOM-32	2	250	500	Control
2	NEMA 17 Stepper Motor	1.7 A, 0.48 N·m	1	350	350	Lifting
3	A4988 Stepper Driver	Pololu A4988	1	120	120	Lifting
4	FA-130RA-18100 DC Motor	3–9 V, ~330 mA	2	80	160	Shooting
5	L298N H-Bridge Module	2 A per channel	1	90	90	Shooting
6	SG90 Micro Servo	1.3 kg·cm, 180°	2	70	140	Sorting
7	TCS3200 Colour Sensor	Full spectrum, 8-pin DIP	1	85	85	Sorting
8	16×4 I2C LCD Display	PCF8574 backpack, 5 V	2	110	220	UI
9	LM2596 Buck Converter	12 V→5 V, 3 A	4	60	240	Power
10	Custom PCB	Power distribution board	1	200	200	Power
11	12 mm Push Buttons	Momentary, 5 V	4	15	60	UI
12	Rocker Switch	SPST, 10 A, 250 V	1	20	20	Power
13	12 V DC Power Adapter	12 V, 5 A minimum	1	180	180	Power
14	PLA Filament (1 kg)	White/Black, 1.75 mm	1	250	250	Mechanical
15	MDF Board 70×60 cm	6 mm thick	1	80	80	Mechanical
16	JST Connectors (sets)	2–4 pin, male/female	10	15	150	Wiring
17	Wiring (mixed gauge)	AWG 22/24, colour coded	1 set	80	80	Wiring
18	M2/M3 Screw Sets	Stainless steel	1 set	30	30	Mechanical
		TOTAL			2,955 EGP	

9. Engineering Calculations

9.1 Actuator Sizing — Sorting Mechanism

The sorting mechanism uses a servo to rotate an arm + gate assembly. The required torque is calculated from rotational inertia and angular acceleration.

Parameter	Value
J_{shaft}	$2.95 \times 10^{-7} \text{ kg}\cdot\text{m}^2$ (from Autodesk Inventor iProperties)
J_{mover}	$1.00 \times 10^{-4} \text{ kg}\cdot\text{m}^2$
J_{arm}	$5.44 \times 10^{-4} \text{ kg}\cdot\text{m}^2$
J_{total} (÷10 safety)	$1.11 \times 10^{-4} \text{ kg}\cdot\text{m}^2$
Angular acceleration θ'	523.2 rad/s^2 (60° rotation in $0.1 \text{ s} \rightarrow 600^\circ/\text{s}$)
Required Torque T_m	$T_m = J \cdot \theta' + T_R = 1.11 \times 10^{-4} \times 523.2 \approx 0.0591 \text{ N}\cdot\text{m} = 0.591 \text{ kg}\cdot\text{cm}$
Resistive Torque T_R	≈ 0 (minimal friction, no external load)
Selected Motor	SG90 Micro Servo — $T_{\text{stall}} = 1.3 \text{ kg}\cdot\text{cm}$
Safety Factor n	$n = 1.3 / 0.591 \approx 2.20 \checkmark$ (>2 acceptable)

9.2 Actuator Sizing — Lifting Mechanism

Parameter	Value
J_{screw}	$0.000551 \text{ kg}\cdot\text{m}^2$
J_{coupler}	$8.52 \times 10^{-7} \text{ kg}\cdot\text{m}^2$
J_{load total}	$5.313 \times 10^{-4} \text{ kg}\cdot\text{m}^2$
J_{motor} (NEMA 17)	$5.7 \times 10^{-6} \text{ kg}\cdot\text{m}^2$
J_{total}	$5.567 \times 10^{-4} \text{ kg}\cdot\text{m}^2$
ω operating	31.41 rad/s (300 rpm nominal)
θ' (spin-up)	314.1 rad/s^2 ($0 \rightarrow 300 \text{ rpm}$ in 0.1 s)
T_{required}	$T_m > J_{\text{tot}} \times \theta' = 5.567 \times 10^{-4} \times 314.1 > 0.175 \text{ N}\cdot\text{m}$
Resistive Torque T_R	≈ 0 (no external weight load on screw flight; balls float on helix)
Selected Motor	NEMA 17 Stepper — $T_{\text{available}} = 0.48 \text{ N}\cdot\text{m}$
Safety Factor n	$n = 0.48 / 0.175 = 2.74 \checkmark$
Operating Speed	$\sim 500 \pm 50 \text{ rpm}$ at constant torque

9.3 Actuator Sizing — Shooting Mechanism

Parameter	Value
Wheel radius r	$32.5 \text{ mm} = 0.0325 \text{ m}$ Wheel mass $m_w \approx 35 \text{ g}$

J_motor	$1.27 \times 10^{-7} \text{ kg}\cdot\text{m}^2$
J_wheel (solid disc)	$J_w = m \cdot r^2 = 0.035 \times (0.0325)^2 = 3.69 \times 10^{-5} \text{ kg}\cdot\text{m}^2$
J_bearing	$7.2 \times 10^{-7} \text{ kg}\cdot\text{m}^2$
J_total	$\approx 3.78 \times 10^{-5} \text{ kg}\cdot\text{m}^2$
Spin-up time	$3 \text{ s} \rightarrow \theta' = 182.5/3 \approx 60.8 \text{ rad/s}^2$
Final ω	$182.5 \text{ rad/s} \rightarrow n = 1,743 \text{ rpm}$
T_required	$T_m = J \times \theta' = 3.78 \times 10^{-5} \times 60.8 \approx 2.3 \times 10^{-3} \text{ N}\cdot\text{m}$
Selected Motor	FA-130RA-18100 with L298N H-bridge and PWM control

9.4 Projectile Calculations

The shooting mechanism must launch a marble from height $h = 0.2 \text{ m}$ below the target, over a horizontal distance $d = 0.8 \text{ m}$, at launch angle $\beta = 30^\circ$.

Parameter	Value
Launch angle β	30°
Horizontal distance d	0.8 m
Height difference h	0.2 m (target above launcher)
Required V_0	$V_0 = (d / \cos \beta) \times \sqrt{(g / 2(d \cdot \tan \beta - h))} = (0.8 / \cos 30^\circ) \times \sqrt{(9.81 / 2(0.8 \cdot \tan 30^\circ - 0.2))} \approx 4.15 \text{ m/s}$
Wheel efficiency η	$\eta \approx 0.7$ (friction coupling)
Required $v_{\text{tangential}}$	$v_{\text{tan}} = V_0 / \eta = 4.15 / 0.7 \approx 5.93 \text{ m/s}$
Required wheel ω	$\omega = v_{\text{tan}} / r = 5.93 / 0.0325 \approx 182.5 \text{ rad/s}$ (1,743 rpm) ✓ matches actuator sizing

9.5 Subsystem Current Consumption

All logic subsystems are supplied at 5 V via buck converters from the 12 V rail. High-power actuators (stepper, DC motors) run directly at 12 V.

Subsystem	Component	V (V)	I typical (mA)	I max (mA)
Control	2× ESP32	3.3 / 5	240	500
Lifting	NEMA 17	12	800	1,700
Lifting driver	A4988	5 (logic)	10	50
Shooting	2× FA-130 DC Motors	12	600	1,200
Shooting driver	L298N	12	36	100
Sorting	2× SG90 Servo	5	400	1,000
Sensing	TCS3200	5	3	5
Display	2× 16×4 I2C LCD	5	40	80

Misc (buttons, LEDs)	–	5	30	50
TOTAL at 12 V rail		12	~2.7 A	~4.7 A

The PTC fuse on the custom PCB is rated at 5 A, providing ~6% headroom above peak load while protecting against short circuits. The 5 A power supply rating is confirmed adequate.

9.6 Power Source Sizing

For battery-powered operation (e.g., exhibitions), a 12 V LiPo battery is sized as follows:

Parameter	Value / Calculation
Avg. operating current	$I_{avg} \approx 2.5 \text{ A}$ (actuators not all on simultaneously; 60% duty cycle assumed)
Desired runtime	3 hours continuous
Required capacity	$Q = I_{avg} \times t = 2.5 \times 3 = 7.5 \text{ Ah}$
Derate for depth of discharge	$Q_{rated} = 7.5 / 0.8 \approx 9.4 \text{ Ah} \rightarrow$ round up to 10 Ah
Recommended battery	12 V, 10 Ah Li-ion / LiFePO4 pack or equivalent adapter
Buck converter efficiency	$\eta \approx 85\%$. I_{12V} for 5 V logic loads: $I = (P_{5V}) / (12 \times 0.85) \approx 1.2 \text{ A}$ additional

10. Safety Considerations

The toy has been designed to comply with general children's toy safety principles applicable to the target age group of 2–10 years:

10.1 Mechanical Safety

- All 3D-printed parts are sanded and deburred post-printing to remove sharp edges and layer lines.
- The Archimedes screw is enclosed in a smooth cylinder — fingers cannot reach the spinning helix.
- Shooting wheel gap is sized for the specific marble diameter. No gap wide enough to trap fingers.
- Marble diameter > 35 mm — above the choking hazard threshold (ISO 8124-1 small parts cylinder test) for children aged 3+. An age label is applied for children under 3.
- All servo horn connections use recessed M2 screws to prevent protrusion injuries.
- No spring-loaded components under high tension accessible to the child.

10.2 Electrical Safety

- All wiring is routed through internal channels in the 3D-printed structure — no exposed conductors reachable by the child.
- JST connectors are keyed and polarised — incorrect reassembly is mechanically prevented.
- The custom PCB includes: reverse-polarity protection (1N5822 diode), over-current protection (5 A PTC resettable fuse), and decoupling capacitors to prevent voltage spikes.
- The 12 V supply uses a CE/UL-certified adapter with insulated barrel connector. No mains voltage enters the toy enclosure.
- All PCB traces are covered with solder mask; PCB is mounted in an enclosed housing within the base.
- The rocker switch is positioned on the rear of the toy, inaccessible during normal forward-facing play.

10.3 Thermal Safety

- Motor drivers (L298N, A4988) are fitted with heatsinks. No surface exceeds 50°C under normal operation.
- All plastic parts are PLA — safe and non-toxic. No ABS or other fume-emitting materials used.
- The PTC fuse provides thermal protection in case of sustained overload, resetting automatically once cooled.

10.4 Software Safety

- A hardware Start/Stop button is always active and takes immediate priority over Bluetooth commands.
- Watchdog timer is enabled on both ESP32 boards — if firmware hangs, the board resets within 8 seconds.
- Stepper and DC motors ramp up gradually (no instantaneous full-power starts) to reduce mechanical shock.

11. Gantt Chart

The Gantt chart below outlines the full project timeline from team organisation (February 2026) through to the final presentation (May 2026). Tasks span six phases: project management, design thinking, mechanical development, electronics and firmware, fabrication, and documentation.



Figure 1 — Gantt Chart for BOM (February–May 2026)

12. Work Breakdown Structure (WBS)

Phase 1 — Project Management & Preliminary Design

- 1.1 Team Organisation: Assign Mechanical Lead, Electronics Lead, Firmware Lead roles.
- 1.2 Planning: Create Gantt Chart to manage the project timeline.
- 1.3 Product Design Specification (PDS): Define technical and functional requirements.
- 1.4 User Research: Conduct research for the 2–10 age group.

Phase 2 — Design Thinking & Ideation

- 2.1 Define the Challenge: Formulate design challenge from user research findings.
- 2.2 Conceptualisation: Generate ideas via mind maps and mood boards.
- 2.3 Sketching: Create initial hand-drawn sketches of the toy concept.
- 2.4 Feedback Loop: Conduct iterative design cycles based on lab feedback.

Phase 3 — Mechanical Design & Analysis

- 3.1.1 Projectile-Based Shooting Mechanism: Design the dual-wheel launcher.
- 3.1.2 Vertical Lifting Mechanism: Design Archimedes screw and NEMA 17 drive.
- 3.1.3 Sorting Mechanism: Design colour-sensor gate system.
- 3.2 CAD Modelling: Create 3D models and assembly in Autodesk Inventor.
- 3.3 Mechanical Calculations: Actuator sizing, projectile motion.
- 3.4 Safety Integration: Ensure all moving parts are safe; verify 100 cm × 100 cm limit.

Phase 4 — Mechatronics & Control Systems

- 4.1 Hardware Selection: Purchase all required hardware.
- 4.2.1 Schematic Design: Circuit diagrams in Proteus / Cirkuit Designer.
- 4.2.2 Custom PCB: Design and manufacture power distribution board.
- 4.2.3 Sensor Integration: Integrate TCS3200 for sorting.
- 4.3.1 Primary Interface: Program LCD displays.
- 4.3.2 Secondary Interface: Develop Bluetooth Processing IO application.
- 4.4 Firmware: Write Arduino/ESP32 code for closed-loop system with ESP-NOW.

Phase 5 — Fabrication & Integration

- 5.1 Modular Assembly: Build toy in modules with dedicated wiring plugs.
- 5.2 Power System: Integrate single 12 V source; verify current budget.
- 5.3 Prototyping: Build physical model using 3D printing and safe materials.
- 5.4 Final Testing: Full system test verifying closed-loop marble pathway.

Phase 6 — Deliverables & Documentation

- 6.1 Technical Documentation: Finalise Technical Report and Bill of Materials.
- 6.2 Media Production: Record project video with voiceover; design A0 poster.
- 6.3 Repository Management: Organise Git repository with correct folder structure.
- 6.4 Final Presentation: Prepare and deliver group presentation.

12. Git Repository

All project files are maintained in a clean Git repository at:

<https://github.com/abdo-A028/MCT231-BASICS-OF-MECHATRONICS>

Repository structure follows the required convention:

Folder	Contents
/mechanical	Autodesk Inventor part files (.ipt), assembly files (.iam), exported drawings (.pdf/.dwg), STL files for 3D printing.
/electrical	Circuit Designer / Proteus schematics, PCB source files (.kicad_pcb or equivalent), Gerber exports, component libraries.
/firmware_mcu1	ESP32 Arduino sketch for Sorting & Lifting controller (MCU1). Includes ESP-NOW transmitter, TCS3200 driver, servo control, LCD library usage.
/firmware_mcu2	ESP32 Arduino sketch for Shooting & Display controller (MCU2). Includes ESP-NOW receiver, A4988 stepper driver, L298N DC motor control.
/app	Processing IO application source code for the Bluetooth remote interface. Includes connection manager, button UI, and serial communication handler.
/docs	Technical report (this document), A0 poster PDF, PDS, test results, Gantt chart, user guide (Kids Guide).

13. A0 Poster

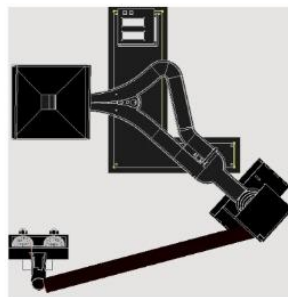


PLANET SORT

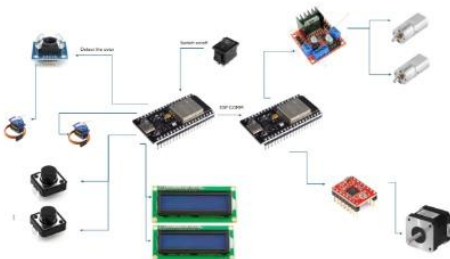
PLANET SORT IS AN AUTOMATED, DUAL-CORE SORTING AND LAUNCH SYSTEM DESIGNED TO CATEGORIZE AND PROPEL PLANETARY OBJECTIVES WITH MECHANICAL PRECISION. DRIVEN BY A SYNCHRONIZED NETWORK OF TWO ESP32 MICROCONTROLLERS COMMUNICATING WIRELESSLY VIA LOW-LATENCY ESP-NOW, THE SYSTEM BALANCES AUTOMATED COLOR-SENSING OPERATIONS WITH IMMEDIATE MANUAL BUTTON OVERRIDES. THIS INTEGRATED MACHINE MANAGES REAL-TIME SYSTEM FEEDBACK ACROSS MULTIPLE I2C DISPLAYS WHILE ORCHESTRATING CONTINUOUS HARDWARE TASKS, INCLUDING A PRECISION STEPPER-DRIVEN LIFTING COLUMN, DYNAMIC ROUTING SERVOS, AND A HIGH-VELOCITY DUAL-MOTOR SHOOTING MECHANISM



PLAN VIEW OF PROJECT



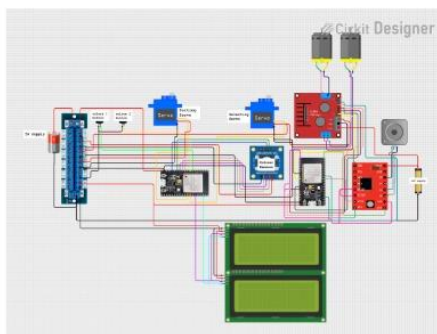
BLOCK DIAGRAM



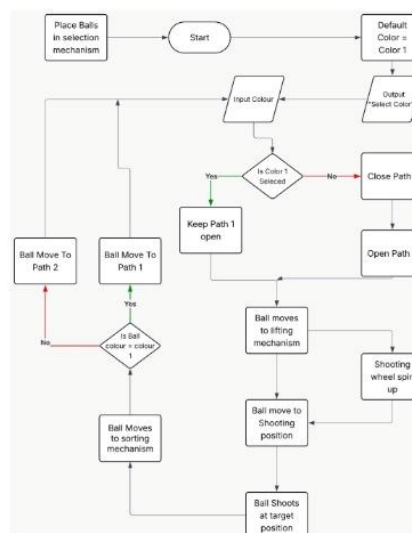
BUILDING PROCESS



ELECTRICAL WIRING



FLOW DIAGRAM



GitHub QR



14. Rules for the toy



How to Play — Quick Guide

WAY 1 — USE THE BUTTONS ON THE TOY

1

POWER ON

Flip the switch to turn the toy on

2

PRESS START

Push the green START button

3

PICK A COLOUR

Press GREEN or RED to choose your planet marble

4

WATCH & PLAY

See the marble sort, lift and launch!

5

PRESS STOP

Push the red STOP button when done

GREEN

← Choose which colour marble to sort!

RED

— OR —

WAY 2 — USE YOUR PHONE (BLUETOOTH)

1

POWER ON

Flip the switch to turn the toy on

2

OPEN THE APP

Open Orbit Quest App on your phone

3

CONNECT

Tap CONNECT — wait for the green tick

4

TAP START

Tap the green START button in the app

5

PICK A COLOUR

Tap GREEN or RED planet in the app

6

TAP STOP

Tap STOP in the app when done

SYSTEM: READY

COLOR: ---

After START

COLOR: GREEN

MODE: MANUAL

Green selected

COLOR: RED

MODE: BT

Red via app

■ The two screens on the toy always show what is happening — watch them!

MCT231 · Ain Shams University × UEL · Orbit Quest Toy

15. Conclusion

The Planet Sort / Orbit Quest project successfully demonstrates a fully integrated mechatronic system designed as a children's toy. The team implemented three distinct mechanisms — colour-based sorting, Archimedes screw vertical lifting, and a dual-wheel projectile shooter — as a continuous, closed-loop marble pathway.

Key engineering achievements include:

- A dual-ESP32 architecture with wireless ESP-NOW communication providing low-latency, router-independent coordination between the sorting/lifting and shooting subsystems.
- Rigorous actuator sizing calculations (safety factors >2.2 for all motors) ensuring reliable performance without oversizing.
- A custom power distribution PCB with reverse-polarity protection, PTC fuse, and modular JST connectors enabling safe and maintainable wiring.
- A complete power budget confirming the 5 A, 12 V supply is adequate with headroom for peak loads, and a 10 Ah battery recommendation for portable operation.
- Full compliance with toy safety principles: no accessible moving parts, no exposed wiring, non-toxic materials, and marble sizing above the choking hazard threshold.

The design process followed the full design thinking methodology from user research and ideation through to physical prototype, documentation, and iterative refinement. All deliverables specified in the MCT231 course requirements have been addressed in this report and the accompanying repository.

The link of the video:

https://drive.google.com/file/d/1nRRADGn482Wg3E0Uluc1cbHO5MsvG_7m/view?usp=sharing

Link for GitHub:

<https://github.com/abdo-A028/MCT231-BASICS-OF-MECHATRONICS>

Appendix: Presentation Slides

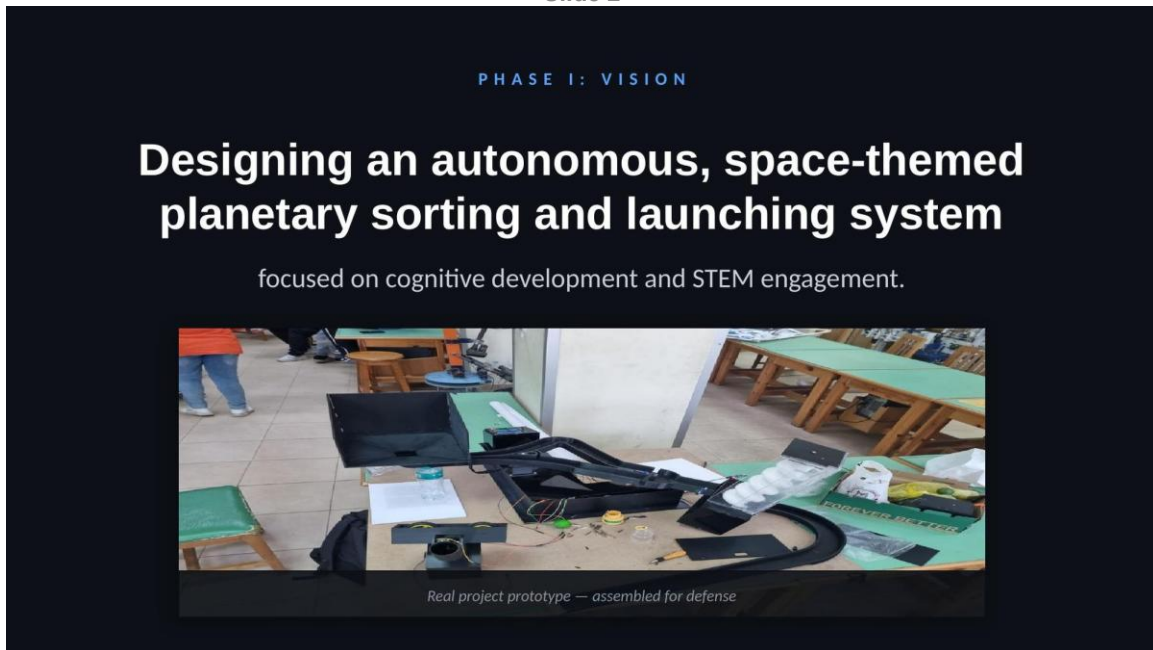
PlanetSort: OrbitQuest — Project Defense Slides (11 Slides)

Slide 1



Slide 1 is the title slide for the presentation. It features a dark background with a blurred image of a workshop. In the top left corner is the Ain Shams University logo, and in the top right corner is the University of East London logo. The main title "PLANETSORT: ORBITQUEST" is displayed in large, white, bold, sans-serif capital letters. Below the title, the subtitle "Advanced Interactive Mechatronic Product Design System" is written in a smaller, white, sans-serif font. In the center, there are two colored circles: a red one labeled "ARES SECTOR RED ORBIT" and a green one labeled "GAEA SECTOR GREEN ORBIT". At the bottom, there are five dark blue rectangular boxes, each containing a team member's name and ID number: Ziad Atef M. Riad (24P0103), Kamal Wahdan (24P0222), Zeyad Amr Nazih (24P0372), Yahya Ashraf (24P0370), and Abdoulrahman Adel (24P0098). At the very bottom, a small line of text reads "Ain Shams University · University of East London · Project Defense 2026".

Slide 2



Slide 2 is titled "PHASE I: VISION" in a small, blue, sans-serif font. Below the title, the main heading "Designing an autonomous, space-themed planetary sorting and launching system" is written in large, white, bold, sans-serif capital letters. Underneath this, a subtitle "focused on cognitive development and STEM engagement." is written in a smaller, white, sans-serif font. At the bottom of the slide, there is a photograph of a real project prototype, which is a complex mechatronic system assembled on a wooden table. The text "Real project prototype — assembled for defense" is written in a small, white, sans-serif font at the bottom of the photograph.

Slide 3

User Research & Target Persona



Primary Audience

Children aged 2-10 years. Focused research indicates high engagement for age 8 through visually stimulating mechanical feedback and goal-oriented tasks.



Core Needs

Visual stimulation (moving parts), color recognition, hand-eye coordination, and a safe, intuitive interface for 'closed-loop' play.

2-10

Target Age Range

3

Core Modules

100×100

cm Footprint

Slide 4

The Design Challenge



Automated Sorting

Autonomous color detection and path routing using optical sensing logic.



Vertical Lifting

Continuous vertical transit via an Archimedes spiral mechanism.



Orbital Launch

High-velocity calibrated projection into a target basketball hoop.

Slide 5

Module I: PlanetScan Sorting



The sorting subsystem identifies planetary marbles by wavelength using the TCS3200 sensor array. Dual SG90 micro-servos actuate high-speed gates to route the spheres.

● Real-time RGB frequency analysis

● Non-contact optical gating

● Dynamic error-correction routing

Slide 6

Module II: Orbital Elevator

Our lifting mechanism utilizes a NEMA 17 stepper motor to drive a 3D-printed helical flight screw within a smooth-walled cylinder.

This module provides precise vertical positioning for the launch bay, ensuring the system is ready for the high-velocity shooting phase.

Motor: NEMA 17 Stepper

Mechanism: Archimedes Spiral

Material: 3D-Printed PLA



Slide 7

Module III: Hyperspace Thruster



The shooting module utilizes twin counter-rotating friction wheels driven by high-RPM DC motors. Calibration ensures repeatable orbital injection.

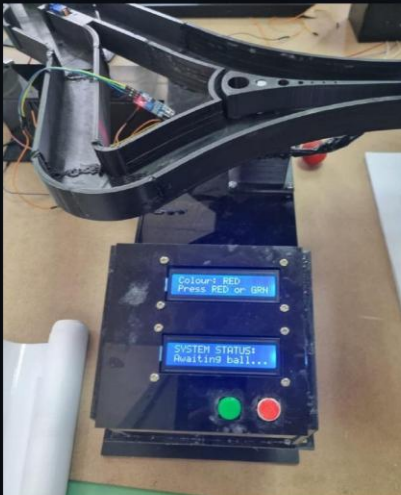
By modulating the PWM duty cycle, we achieve the precise initial velocity required to overcome gravity and reach the target hoop.

4.15 m/s

Calibrated Launch Velocity · $\beta = 30^\circ$

Slide 8

Master-Slave Control Logic



● ESP32 Master

Handles high-level tasks: Color sensing, TFT display, Bluetooth telemetry, and ESP-NOW communication with the slave controller.

● Arduino Uno Slave

Dedicated to low-level actuation: Precise stepper pulses, DC motor PWM curves, and servo timing isolation.

Communication via ESP-NOW wireless protocol - 2.4 GHz band

Slide 9

Dual Interfaces: Mission Control



LCD Control Panel

Local onboard display · Colour detection readout
Green & Red button input



Ball Sorter App

Bluetooth remote control · Score tracking
Shoot speed slider · RGB sensor raw data

Slide 10

Engineering Validation

$$v_o = \frac{d}{\cos(\beta)} \times \sqrt{\frac{g}{2(d \cdot \tan(\beta) - h)}} \approx 4.15 \text{ m/s}$$

Parameter	Value	Validation Status
Initial Velocity (v_o)	4.15 m/s	Calculated & Tested
Launch Angle (β)	30°	Fixed Mechanical Constraint
Current Draw (Max)	< 5.0 A	Fused Safety Limit
Motor Safety Factor	> 2.2	Industry Standard Compliant

Slide 11

Safety & Compliance Protocols

● Concealed Architecture

All electrical wiring routed through internal structural channels to remain completely hidden and safe.

● Thermal Safety

Onboard resettable 5A PTC fuse safeguards logic and motor rails against stall-current overloads and overheating.

● Child-Safe Materials

Non-toxic PLA and smoothed MDF surfaces eliminate sharp edges and chemical exposure risks.

● Hardware Interrupt

A physical master-stop button provides immediate electrical isolation of all high-power actuators.

OrbitQuest: Launching Curiosity into STEM · Ain Shams University & University of East London · Project Defense 2026